Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Кафедра програмування

Лабораторна робота №7

РОЗПАРАЛЕЛЕННЯ АЛГОРИТМУ ПРИМА

з курсу “Паралельні та розподілені обчислення”

Виконав:

студент групи ПМІ-33

Яценко Владислав Тарасович

Львів – 2025

Мета роботи

Мета цієї роботи – реалізувати алгоритм Прима для побудови мінімального кісткового дерева у зваженому зв'язному неорієнтованому графі двома способами:

- послідовно, обробляючи вершини графа одна за одною;
- паралельно, коли граф розбивається на кілька частин, і кожна частина обчислюється окремим потоком.

Теоретична частина

Алгоритм Прима

Алгоритм Прима - це жадібний алгоритм для побудови мінімального кісткового дерева (МКД) у зваженому зв'язному неорієнтованому графі. Мінімальне кісткове дерево - це підграф, який з'єднує всі вершини з мінімальною сумарною вагою ребер.

Основні кроки:

- На кожному кроці вибирається вершина з мінімальною вагою ребра, що з'єднує її з уже побудованим деревом
- Ця вершина додається до дерева
- Оновлюються мінімальні ваги для всіх сусідніх вершин
- Процес повторюється n разів (де n - кількість вершин)

Послідовна реалізація

Послідовна версія обробляє граф елемент за елементом без паралелізму. На кожній ітерації:

1. Знаходиться вершина з мінімальним ключем серед невідвіданих
2. Вершина додається до МКД
3. Оновлюються ключі всіх сусідніх вершин

Кількість операцій: приблизно n^2 операцій для пошуку мінімуму та оновлення ключів.

Паралельна реалізація

Ідея паралельної реалізації:

- Розбиваємо вершини графа на k частин (де k - кількість потоків)
- Кожен потік обробляє свій діапазон вершин (близько n/k вершин)
- Кожен потік знаходить локальний мінімум у своєму діапазоні
- Головний потік визначає глобальний мінімум з усіх локальних
- Використовуємо Barrier для синхронізації потоків на кожному кроці
- Кожен потік оновлює ключі вершин у своєму діапазоні

Переваги використання Barrier:

- Мінімальні накладні витрати на синхронізацію
- Потоки не перестворюються на кожному кроці
- Значно вища продуктивність на великих графах

Прискорення паралельного алгоритму:

$$S = \frac{T_{\text{посл}}}{T_{\text{парал}}}$$

Ефективність:

$$E = \frac{S}{k} \cdot 100\%$$

Чим більше потоків, тим швидше зазвичай програма працює, але ефективність падає через накладні витрати на створення та керування потоками.

Результати експериментів

Експеримент 1: Граф розміром 10 000 вершин

Параметри:

- Розмір графа: 10 000 × 10 000 вершин
- Стартова вершина: 0
- Максимальна кількість потоків: 8

Результат:

```
Введіть розмір графа (кількість вершин n): 10000
Введіть максимальну кількість потоків (k): 8
Введіть стартову вершину a (0 до 9999): 0

Генерація графа та обчислення...

==============================================================================
|  Потоки    |  Час виконання (мс)   |   Прискорення   |   Ефективність (%)   |
==============================================================================
|          1 |                   621 |          1,0000 |              100,00% |
|          2 |                   531 |          1,1695 |               58,47% |
|          3 |                   532 |          1,1673 |               38,91% |
|          4 |                   477 |          1,3019 |               32,55% |
|          5 |                   516 |          1,2035 |               24,07% |
|          6 |                   499 |          1,2445 |               20,74% |
|          7 |                   719 |          0,8637 |               12,34% |
|          8 |                   857 |          0,7246 |                9,06% |
==============================================================================
  a112 ---- a1  (вага: 1)
  a18 ---- a2  (вага: 1)
  a158 ---- a3  (вага: 1)
  a181 ---- a4  (вага: 1)
  a175 ---- a5  (вага: 1)
  a85 ---- a6  (вага: 1)
  a188 ---- a7  (вага: 1)
  a86 ---- a8  (вага: 1)
  a1 ---- a9  (вага: 1)
  a135 ---- a10  (вага: 1)
  ... та ще 9989 ребер
```

```
===     Аналіз продуктивності     ===

1. Параметри графа:
    1.1 Розмір: 10000 × 10000 вершин
    1.2 Стартова вершина: a0

2. Час виконання:
    2.1 Без потоків (послідовно, 1 потік): 621 мс
    2.2 З 4 потоками: 477 мс

3.  Найкращий результат:
    3.1 Оптимальна кількість потоків: 4
    3.2 Прискорення: 1,3019x
    3.3 Ефективність: 32,55%
```

Найкращий результат:

- Оптимальна кількість потоків: 4
- Час виконання: 477 мс
- Прискорення: 1,3019x
- Ефективність: 32,55%

Висновок: На графах середнього розміру оптимум досягається при 4 потоках. Подальше збільшення кількості потоків (7-8) призводить до погіршення продуктивності через значні накладні витрати на синхронізацію.

Експеримент 2: Граф розміром 50 000 вершин

Параметри:

- Розмір графа: 50 000 × 50 000 вершин
- Стартова вершина: 0
- Максимальна кількість потоків: 8

Результат:

```
Введіть розмір графа (кількість вершин n): 50000
Введіть максимальну кількість потоків (k): 8
Введіть стартову вершину a (0 до 49999): 0

Генерація графа та обчислення...

==============================================================================
|  Потоки    |  Час виконання (мс)   |   Прискорення   |   Ефективність (%)   |
==============================================================================
|          1 |                 21539 |          1,0000 |              100,00% |
|          2 |                 13544 |          1,5903 |               79,51% |
|          3 |                 11289 |          1,9080 |               63,60% |
|          4 |                  8522 |          2,5275 |               63,19% |
|          5 |                  9831 |          2,1909 |               43,82% |
|          6 |                 10313 |          2,0885 |               34,81% |
|          7 |                 11004 |          1,9574 |               27,96% |
|          8 |                 13282 |          1,6217 |               20,27% |
==============================================================================
  a82 ---- a1  (вага: 1)
  a75 ---- a2  (вага: 1)
  a51 ---- a3  (вага: 1)
  a136 ---- a4  (вага: 1)
  a74 ---- a5  (вага: 1)
  a196 ---- a6  (вага: 1)
  a303 ---- a7  (вага: 1)
  a54 ---- a8  (вага: 1)
  a60 ---- a9  (вага: 1)
  a114 ---- a10  (вага: 1)
  ... та ще 49989 ребер
```

```
===     Аналіз продуктивності     ===

1. Параметри графа:
    1.1 Розмір: 50000 × 50000 вершин
    1.2 Стартова вершина: a0

2. Час виконання:
    2.1 Без потоків (послідовно, 1 потік): 21539 мс
    2.2 З 4 потоками: 8522 мс

3.  Найкращий результат:
    3.1 Оптимальна кількість потоків: 4
    3.2 Прискорення: 2,5275x
    3.3 Ефективність: 63,19%
```

Найкращий результат:

- Оптимальна кількість потоків: 4
- Час виконання: 8522 мс
- Прискорення: 2,5275x
- Ефективність: 63,19%

Висновок: На великих графах паралелізація дає значне прискорення - майже в 2,5 рази при використанні 4 потоків. Оптимальна кількість потоків залишається 4, що забезпечує найкращий баланс між прискоренням та ефективністю. Ефективність значно вища порівняно з меншими графами (63,19% проти 32,55%).

Висновок до лабораторної роботи:

Паралельна реалізація алгоритму Прима дозволяє досягти значного прискорення на великих графах при оптимальній кількості потоків. Досліджено, що оптимальна кількість потоків становить 4 для обох тестових конфігурацій.